



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING · 25CS2104E

2026–27, Term I · Section 8 · Team 1

Alarm Clock System

A signal-driven, concurrent alarm-scheduling utility built on core Linux/POSIX process, signal, memory, file and threading primitives.

FACULTY IN-CHARGE

Dr. Muniraju Naidu

TEAM 1 · TIRAMISU

Teja Hasini Reddy (2520030048)

Hansika Reddy (2520030237)

Simran (2520030166)

Anthony Nissy (2520030064)

What This Project Builds

A Linux-based alarm clock, written in C, that lets a user schedule one or more alarms — by absolute time or delay — while the main process stays free to accept further commands. The project traces a request's journey from user space to kernel space and back.



Process per Alarm

`fork()` creates a dedicated child process to manage each scheduled alarm independently.



Async Notification

`alarm()`/POSIX timers request a kernel `SIGALRM`, caught by a `sigaction()` handler.



Dynamic Tracking

A `malloc()/free()`-based linked list tracks, matches and cancels active alarms.



Thread-Safe Monitor

A POSIX thread with mutex + condition variable guards the shared alarm list.

Every alarm event — creation, firing, cancellation, and errors — is logged to a file using low-level Linux file I/O system calls.

THE CHALLENGE

Problem Statement

Linux systems constantly need to trigger time-based actions — reminders, timeouts, scheduled tasks — without freezing everything else while they wait.

The core inefficiency: repeatedly polling the clock (busy-waiting) burns CPU cycles and breaks down once several independent alarms must be tracked at once.



Suspend a process efficiently until a precise future instant



Receive an asynchronous kernel notification the moment it arrives



Manage multiple independent alarm requests without conflict



Allow an active alarm to be cancelled safely, on demand

All using standard Linux/POSIX process, signal, memory and concurrency primitives — not GUI or polling-based logic.



Project Objectives

01

Schedule with Precision

Accept an absolute time or a relative delay from the user and schedule the alarm using POSIX system calls.

02

Notify Asynchronously

Spawn a dedicated process per alarm with `fork()`, and use `alarm()/POSIX` timers with a `sigaction()`-installed `SIGALRM` handler for async notification.

03

Manage & Cancel

Track multiple simultaneous alarm requests in a dynamic linked list and let the user cancel any active alarm before it fires.

04

Synchronize & Log

Coordinate alarm monitoring with POSIX threads, mutexes and condition variables, and maintain a persistent log via Linux file I/O.

Proposed Methodology



1

Menu-Driven Input

Main process accepts an alarm time or delay in seconds and computes the wait interval.



2

Insert into Alarm List

Request is stored as a dynamically allocated node in the active-alarms linked list.



3

Fork a Child Process

Parent forks a dedicated child to own this alarm while continuing to accept new requests.



4

Arm the Timer

Child calls `alarm()` / `timer_create()+timer_settime()`, then suspends itself with `pause()`.



5

SIGALRM Delivered

At the target instant the kernel sends SIGALRM, intercepted by a `sigaction()` handler.



6

Notify, Reap & Log

Handler updates status and exits the child; parent reaps it via `waitpid()`; event is logged.

Validation: strace confirms the system-call sequence; gdb and Valgrind verify correct process behavior and leak-free memory.

OS Concepts & Linux APIs Used



User ↔ Kernel Boundary

System calls & mode switching move the alarm request and SIGALRM delivery across the boundary.



Process Management

fork() creates per-alarm processes; wait()/waitpid() reap them and prevent zombies.



Signals

alarm()/POSIX timers plus a sigaction()-based SIGALRM handler drive asynchronous notification.



Dynamic Memory

malloc()/realloc()/free() build the alarm linked list; Valgrind checks for leaks.



Concurrency

pthread_create()/pthread_join() with mutexes & condition variables synchronize the alarm list.



File I/O

open(), write(), close() and file descriptors persist a log of every alarm event.

Tools & Platforms Used



Linux / Ubuntu

Development and execution platform providing all Linux/POSIX system calls used.



C / C++

Implementation language for process, signal, memory, file and thread logic.



GCC

Compiler used to build the system, driven through a project Makefile.



GDB & strace

Debugging and system-call tracing tools to verify process, signal and I/O behavior.



Valgrind

Detects memory leaks and errors in the dynamically allocated alarm data structures.



Makefile

Automates compilation and linking of all project source files.

Individual Contributions

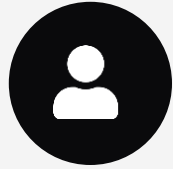


Teja Hasini Reddy

2520030048

Input & Scheduling

Designed the input-handling module and alarm-scheduling logic; implemented fork()-based child process creation for each alarm request.

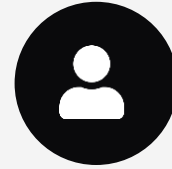


Hansika Reddy

2520030237

Signals & Cancellation

Implemented signal generation and handling (alarm(), sigaction(), SIGALRM handler) and the alarm-cancellation feature.

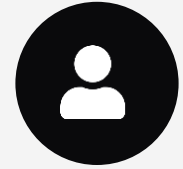


Simran

2520030166

Memory Management

Implemented dynamic memory management for the active-alarm linked list and carried out Valgrind-based memory testing.



Anthony Nissy

2520030064

Threading & Logging

Implemented POSIX-thread-based alarm monitoring with mutex/condition-variable synchronization, file-based event logging, and test documentation.

Expected Outcome

- Reliably schedules and fires single or multiple alarms at the correct time using signal-based notification instead of CPU-wasting busy-waiting.
- Lets the user view and cancel pending alarms at any point during execution.
- Terminates all child processes cleanly, leaving zero zombie processes.
- Produces a readable event log confirming alarm creation, firing and cancellation.
- Ships strace traces and a Valgrind report proving correct system-call usage and leak-free memory.



GitHub Repository

[7Hansika7/Tiramisu_OSSP](#)

Deliverables: source code, Makefile, README, test cases & output evidence.



THANK YOU

Alarm Clock System · Team 1, Section 8 · Tiramisu

Questions & feedback welcome

